

```

import { blob } from "https://esm.town/v/std/blob";

const cors = { "Access-Control-Allow-Origin": "*", "Access-Control-Allow-Headers"

function todayLocal() {
    const d = new Date();
    return d.getFullYear() + "-" + String(d.getMonth()+1).padStart(2,"0") + "-" + S
}

function dateFromSyncedAt(syncedAtStr) {
    // syncedAt is UTC from the iPhone. Use Intl to get the date in the
    // timezone that HAE is reporting from - we detect it from the offset
    // in any metric date string, falling back to UTC-based guess
    if (!syncedAtStr) return null;
    const d = new Date(syncedAtStr);
    if (isNaN(d.getTime())) return null;
    // Use Intl.DateTimeFormat with UTC and check - the phone sets syncedAt
    // so we trust it as close to real local time
    // Return as-is date in UTC - caller will compare with metric dates
    return null; // handled below
}

function getLocalDateFromMetrics(metrics) {
    // Extract local date from HAE data timestamps which include timezone offset
    // HAE sends dates like "2026-04-26 22:13:00 -0700"
    // We parse the date portion directly (before the time) which is already local
    for (const m of metrics) {
        if (m.data && m.data.length > 0) {
            for (const e of m.data) {
                const dateStr = e.date || e.startDate || "";
                // HAE date format: "YYYY-MM-DD HH:MM:SS ±HHMM"
                const match = dateStr.match(/^(\\d{4}-\\d{2}-\\d{2})/);
                if (match) {
                    // Convert the timestamp with offset to get true local date
                    const d = new Date(dateStr.replace(" ", "T").replace(/(\\+|\\-)(\\d{2})(\\d{
                    if (!isNaN(d.getTime())) {
                        // Use the offset embedded in the string to get local date
                        const offsetMatch = dateStr.match(/([+-])(\\d{2}):?(\\d{2})$/);
                        if (offsetMatch) {
                            const sign = offsetMatch[1] === "+" ? 1 : -1;
                            const offsetMins = sign * (parseInt(offsetMatch[2]) * 60 + parseInt
                            const localMs = d.getTime() + offsetMins * 60000;
                            const localDate = new Date(localMs);
                            return localDate.getUTCFullYear() + "-" +

```

```

        String(localDate.getUTCMonth()+1).padStart(2,"0") + "-" +
        String(localDate.getUTCDate()).padStart(2,"0");
    }
    return match[1]; // fallback to date portion
}
}
}
}
}
return null;
}

function round(v, dp) {
    const f = Math.pow(10, dp);
    return Math.round(v * f) / f;
}

function extractMetric(metrics, name, type, date) {
    const m = metrics.find(function(x) { return x.name === name; });
    if (!m || !m.data) return null;
    let entries = m.data;
    if (date) entries = entries.filter(function(e) {
        const ed = (e.date || e.startDate || "").trim();
        // Match "YYYY-MM-DD" at start, handle both "YYYY-MM-DD HH:MM:SS" and ISO for
        return ed.startsWith(date) || ed.slice(0,10) === date;
    });
    if (!entries.length) return null;
    const vals = entries.map(function(e) { return parseFloat(e.qty || e.value || 0)
    if (!vals.length) return null;
    if (type === "sum") return vals.reduce(function(a,b) { return a+b; }, 0);
    if (type === "avg") return vals.reduce(function(a,b) { return a+b; }, 0) / vals
    if (type === "last") return vals[vals.length - 1];
    return vals[0];
}

function parseSleep(metrics, today) {
    const m = metrics.find(function(x) { return x.name === "sleep_analysis"; });
    if (!m || !m.data || !m.data.length) return { hours: null, bedtime: null, wake:

    // HAE sends sleep as objects with totalSleep, sleepStart, sleepEnd, date field
    const ydDate = (function() {
        const d = new Date();
        d.setDate(d.getDate()-1);
        return d.getFullYear() + "-" + String(d.getMonth()+1).padStart(2,"0") + "-" +
    })();

    // Sort by date descending, find most recent entry

```

```

const sorted = m.data.slice().sort(function(a,b) {
  return (b.date||"").localeCompare(a.date||"");
});

// Try to find entry for today or yesterday
let entry = sorted.find(function(e) { return (e.date||"").startsWith(today); })
if (!entry) entry = sorted.find(function(e) { return (e.date||"").startsWith(yd
if (!entry) entry = sorted[0]; // use most recent
if (!entry) return { hours: null, bedtime: null, wake: null, date: null };

// Extract hours from totalSleep field
const hrs = parseFloat(entry.totalSleep || entry.asleep || 0);
if (!hrs || hrs <= 0) return { hours: null, bedtime: null, wake: null, date: nu

// Extract bedtime and wake from sleepStart/sleepEnd
function fmtTime(s) {
  if (!s) return null;
  // HAE format: "2026-04-21 23:02:57 -0700"
  // Extract time and offset directly without Date parsing
  const m = s.match(/(\d{2}):(\d{2}):\d{2}\s*([+-])(\d{2}):?(\d{2})/);
  if (m) {
    // m[1]=hours, m[2]=mins, m[3]=sign, m[4]=offH, m[5]=offM
    // The time shown is already local time in the offset timezone
    return m[1] + ":" + m[2];
  }
  // Fallback
  const d = new Date(s.replace(/ ([+-])/, "T$1").replace(/(\d{2})(\d{2})$/, "$1
  if (isNaN(d.getTime())) return null;
  return String(d.getHours()).padStart(2,"0") + ":" + String(d.getMinutes()).pa
}

const sleepDate = (entry.date||"").slice(0,10);
return {
  hours: round(hrs, 2),
  bedtime: fmtTime(entry.sleepStart || entry.inBedStart),
  wake: fmtTime(entry.sleepEnd || entry.inBedEnd),
  date: sleepDate
};
}

export default async function handler(req) {
  const url = new URL(req.url);

  if (req.method === "OPTIONS") {
    return new Response("ok", { headers: cors });
  }

```

```

if (req.method === "GET" && url.pathname.includes("/metrics")) {
  try {
    const data = await blob.getJSON("last_raw_metrics");
    if (!data) return new Response(JSON.stringify({ error: "No raw data yet" }));
    return new Response(JSON.stringify(data), { headers: { ...cors, "Content-Type"
  } catch(e) {
    return new Response(JSON.stringify({ error: e.message }), { status: 500, he
  }
}

if (req.method === "GET" && url.pathname.includes("/food/search")) {
  const q = url.searchParams.get("q") || "";
  try {
    const r = await fetch("https://world.openfoodfacts.org/cgi/search.pl?search
    const d = await r.json();
    return new Response(JSON.stringify(d), { headers: { ...cors, "Content-Type"
  } catch(e) {
    return new Response(JSON.stringify({ error: e.message }), { status: 500, he
  }
}

if (req.method === "GET" && url.pathname.includes("/food/barcode")) {
  const code = url.searchParams.get("code") || "";
  try {
    const r = await fetch("https://world.openfoodfacts.net/api/v0/product/" + e
    const d = await r.json();
    return new Response(JSON.stringify(d), { headers: { ...cors, "Content-Type"
  } catch(e) {
    return new Response(JSON.stringify({ error: e.message }), { status: 500, he
  }
}

if (req.method === "GET") {
  try {
    const data = await blob.getJSON("last_cleaned");
    if (!data) return new Response(JSON.stringify({ error: "No data yet" })), {
    return new Response(JSON.stringify(data), { headers: { ...cors, "Content-Type"
  } catch(e) {
    return new Response(JSON.stringify({ error: e.message }), { status: 500, he
  }
}

if (req.method === "POST") {
  try {
    const parsed = await req.json();
    let today = todayLocal();

```

```

if (parsed.data && parsed.data.metrics) {
  const metrics = parsed.data.metrics;
  // Get local date from HAE metric date strings (already in local time)
  const localDate = getLocalDateFromMetrics(metrics);
  if (localDate) {
    today = localDate;
  } else {
    // Fallback: derive from syncedAt by finding the most recent metric dat
    let latestMetricDate = '';
    for (const m of metrics) {
      if (m.data) for (const e of m.data) {
        const ds = (e.date || '').slice(0,10);
        if (ds > latestMetricDate) latestMetricDate = ds;
      }
    }
    if (latestMetricDate) today = latestMetricDate;
  }

  await blob.setJSON("last_raw_metrics", { names: metrics.map(function(m) {

function getM(name, type) {
  // NEVER fall back to null date - that would sum all historical data
  const v = extractMetric(metrics, name, type, today);
  return v !== null ? round(v, 2) : null;
}

const weightLbs = extractMetric(metrics, "weight_body_mass", "avg", today)
const sleep = parseSleep(metrics, today);
// Check unit from HAE - iPhone set to miles sends "mi", km sends "km"
function getMetricUnit(metrics, name) {
  const m = metrics.find(x => x.name === name);
  if (m && m.units) return m.units.toLowerCase();
  if (m && m.data && m.data[0] && m.data[0].unit) return m.data[0].unit.t
  return "km"; // default assume km
}
const walkUnit = getMetricUnit(metrics, "walking_running_distance");
const cycleUnit = getMetricUnit(metrics, "cycling_distance");
// Max source for distance too - avoids Watch + iPhone double count
const walkRaw = (function(){
  const m = metrics.find(function(x){return x.name==="walking_running_dis
  if(!m||!m.data) return 0;
  const entries = m.data.filter(function(e){return (e.date||"").startsWith
  if(!entries.length) return 0;
  const bySource = {};
  entries.forEach(function(e){const src=e.source||"default";bySource[src]
  return Math.max.apply(null, Object.values(bySource));
}))( ) || 0;

```

```

const cycleRaw = extractMetric(metrics, "cycling_distance", "sum", today)
// Convert to miles only if unit is km
const walkMiles = round(walkUnit === "mi" ? walkRaw : walkRaw * 0.621371,
const cycleMiles = round(cycleUnit === "mi" ? cycleRaw : cycleRaw * 0.621
const distMiles = round(walkMiles + cycleMiles, 2);
const standHours = extractMetric(metrics, "apple_stand_hour", "sum", toda
// Blood pressure
// Blood pressure - HAE sends {systolic, diastolic} per entry
const bpM = metrics.find(x => x.name === "blood_pressure");
const bpEntry = bpM && bpM.data && bpM.data.length ?
  bpM.data.slice().sort((a,b) => b.date.localeCompare(a.date))[0] : null;
const bpSys = bpEntry ? Math.round(bpEntry.systolic||0)||null : null;
const bpDia = bpEntry ? Math.round(bpEntry.diastolic||0)||null : null;
const exerciseMins = extractMetric(metrics, "apple_exercise_time", "sum",
const activeEnergy = extractMetric(metrics, "active_energy", "sum", today
const restingEnergy = extractMetric(metrics, "basal_energy_burned", "sum"
const totalBurned = restingEnergy ? Math.round(activeEnergy + restingEner

const cleaned = {
  date: today,

  // Use max single entry for steps - avoids double-counting Watch + iPho
  // HAE sends cumulative totals per interval, max gives highest cumulati
  stepsToday: (function(){
    const m = metrics.find(function(x){return x.name==="step_count";});
    if(!m||!m.data) return null;
    const entries = m.data.filter(function(e){return (e.date||"").startsW
    if(!entries.length) return null;
    // Group by source, sum each source, take the max source total
    const bySource = {};
    entries.forEach(function(e){
      const src = e.source || "default";
      bySource[src] = (bySource[src]||0) + parseFloat(e.qty||e.value||0);
    });
    const maxSteps = Math.max.apply(null, Object.values(bySource));
    return Math.round(maxSteps)||null;
  })(),
  burned: Math.round(activeEnergy*10)/10,
  restingEnergy: restingEnergy,
  totalBurned: totalBurned,
  rhr: getM("resting_heart_rate", "avg"),
  hrv: getM("heart_rate_variability", "avg"),
  spo2: getM("blood_oxygen_saturation", "avg"),
  rr: getM("respiratory_rate", "avg"),
  sleepLast: sleep.hours,

  sleepBedtime: sleep.bedtime,

```

```

        sleepWake: sleep.wake,
        sleepDate: sleep.date,
        weightKg: weightLbs ? round(weightLbs / 2.20462, 3) : null,
        distMiles: distMiles || null,

        walkMiles: walkMiles || null,
        cycleMiles: cycleMiles || null,
        standHours: standHours,
        bpSystolic: bpSys,
        bpDiastolic: bpDia,
        exerciseMins: exerciseMins,
        syncedAt: new Date().toISOString(),
        source: "health-auto-export"
    };

    await blob.setJSON("last_cleaned", cleaned);
    return new Response(JSON.stringify({ ok: true, date: today }), { headers:
    }

    return new Response(JSON.stringify({ error: "No metrics found" }), { status
    } catch(e) {
        return new Response(JSON.stringify({ error: e.message }), { status: 500, he
        }
    }

    return new Response("Not found", { status: 404, headers: cors });
}

```